

◆ WEEK 2 — SESSION 6

DATABASES

[Amazon RDS vs DynamoDB](#)

GitHub Lab — Database Architecture Foundations

◆ LAB TITLE

Database Architecture Lab

[Amazon RDS + DynamoDB Foundations](#)

◆ LAB OBJECTIVE

By the end of this lab, students will:

- ✓ Understand relational vs NoSQL databases
 - ✓ Explore Amazon RDS and DynamoDB
 - ✓ Build foundational database awareness
 - ✓ Learn architecture decision thinking
 - ✓ Document database strategy professionally on GitHub
-

◆ REQUIRED AWS SERVICES

Students will interact with:

- Amazon RDS
 - Amazon DynamoDB
 - AWS Console
 - IAM
 - CloudWatch (observation only)
-

◆ REQUIRED GITHUB STRUCTURE

Students must update their repository:

```
aws-cloud-practitioner-bootcamp/  
└─ Week-2-Databases/  
    ├── database-comparison.md  
    ├── use-case-mapping.md  
    ├── rds-observations.md  
    ├── dynamodb-observations.md  
    ├── architecture-thinking.md  
    └── screenshots/
```

◆ LAB 1 — AMAZON RDS EXPLORATION

🎯 Objective

Understand how managed relational databases work in AWS.

◆ TASKS

Step 1 — Open Amazon RDS

Inside AWS Console:

AWS Console → RDS

Observe:

- ✓ Database dashboard
 - ✓ Engine options
 - ✓ Multi-AZ options
 - ✓ Backup settings
 - ✓ Monitoring features
-

Step 2 — Investigate Supported Engines

Document:

- MySQL

- PostgreSQL
 - MariaDB
 - Oracle
 - SQL Server
-

Step 3 — Analyze Configuration Areas

Observe:

- ✓ Instance class
 - ✓ Storage configuration
 - ✓ Security settings
 - ✓ Network placement
 - ✓ Backup retention
-

◆ REQUIRED GITHUB DOCUMENT

File:

`rds-observations.md`

Students must explain:

- What Amazon RDS does
 - Why businesses use relational databases
 - What AWS manages automatically
 - Why backups matter
 - Why high availability matters
-

◆ LAB 2 — DYNAMODB EXPLORATION

🎯 Objective

Understand NoSQL and serverless database architecture.

◆ TASKS

Step 1 — Open DynamoDB

Inside AWS Console:

AWS Console → DynamoDB

Observe:

- ✓ Tables
 - ✓ Partitions
 - ✓ Capacity modes
 - ✓ Global tables
 - ✓ Backup features
-

Step 2 — Investigate DynamoDB Features

Students must explore:

- On-demand scaling
 - Key-value structure
 - Performance characteristics
 - Serverless operations
-

Step 3 — Compare Interface Differences

Students should compare:

RDS vs DynamoDB interface design.

Questions:

- Which feels more structured?
 - Which appears more scalable?
 - Which seems easier for rapid growth?
-

◆ REQUIRED GITHUB DOCUMENT

File:

dynamodb-observations.md

Students must explain:

- What DynamoDB is
 - Why it is considered NoSQL
 - Why it is called serverless
 - What workloads benefit from DynamoDB
-

◆ LAB 3 — RELATIONAL VS NOSQL THINKING

Objective

Develop architecture decision-making awareness.

◆ TASK

Students must create:

`database-comparison.md`

REQUIRED COMPARISON TABLE

Feature	Amazon RDS	DynamoDB
Database Type		
Structure		
Scaling Style		
Performance Focus		
Best Workloads		
Example Use Cases		

◆ REQUIRED THINKING

Students must answer:

1.

Why would a banking system prefer relational databases?

2.

Why would a gaming application prefer DynamoDB?

3.

What happens if the wrong database is selected?

4.

How does scalability affect database decisions?

◆ LAB 4 — USE CASE MAPPING

🎯 Objective

Match business scenarios to correct database solutions.

◆ TASK

Students must create:

`use-case-mapping.md`

◆ REQUIRED SCENARIOS

Students must decide whether to use:

- ✓ Amazon RDS
 - OR
 - ✓ DynamoDB
-

Scenario 1

Bank transaction system

Scenario 2

Mobile gaming leaderboard

Scenario 3

Hospital patient records

Scenario 4

Real-time IoT sensor platform

Scenario 5

Large e-commerce product catalog

◆ IMPORTANT REQUIREMENT

Students must explain:

- ✓ WHY they selected the database
 - ✓ scalability considerations
 - ✓ performance considerations
 - ✓ structure requirements
-

◆ LAB 5 — ARCHITECTURE THINKING

🎯 Objective

Develop cloud architecture reasoning.

◆ TASK

Students create:

`architecture-thinking.md`

◆ REQUIRED QUESTIONS

1.

What risks exist when selecting the wrong database?

2.

Why must cloud architects understand workloads before deployment?

3.

How do performance requirements affect architecture decisions?

4.

Why is scalability important in cloud systems?

5.

What is the difference between:

- structured data
 - flexible data
-

◆ REQUIRED SCREENSHOTS

Students must upload:

/screenshots/

Required screenshots:

- ✓ Amazon RDS dashboard
 - ✓ DynamoDB dashboard
 - ✓ Database engine options
 - ✓ DynamoDB table configuration
 - ✓ AWS Console observations
-

◆ GITHUB EXPECTATIONS

Students must maintain:

- ✓ Professional README formatting
 - ✓ Clear markdown structure
 - ✓ Proper headings
 - ✓ Clean screenshots
 - ✓ Architecture reasoning
 - ✓ Governance thinking
-

◆ COMMIT EXPECTATIONS

Good commits:

Added RDS comparison analysis
Documented DynamoDB observations
Completed database decision scenarios
Uploaded AWS database screenshots
Updated architecture-thinking notes

Bad commits:

```
update  
new work  
database stuff
```

◆ SECURITY & GOVERNANCE RULES

Students must NEVER upload:

- ✗ AWS credentials
 - ✗ Access keys
 - ✗ Secret keys
 - ✗ Sensitive information
-

◆ FINAL DELIVERABLE

By the end of this lab, students should demonstrate:

- ✓ Understanding of RDS
 - ✓ Understanding of DynamoDB
 - ✓ Relational vs NoSQL reasoning
 - ✓ Architecture decision thinking
 - ✓ Professional documentation habits
 - ✓ Cloud governance awareness
-

◆ FINAL PRINCIPLE

“Strong cloud engineers know services.

Great cloud architects know WHEN and WHY to use them.”